Konsep Inheritance

IF2210 – Semester II 2025/2026

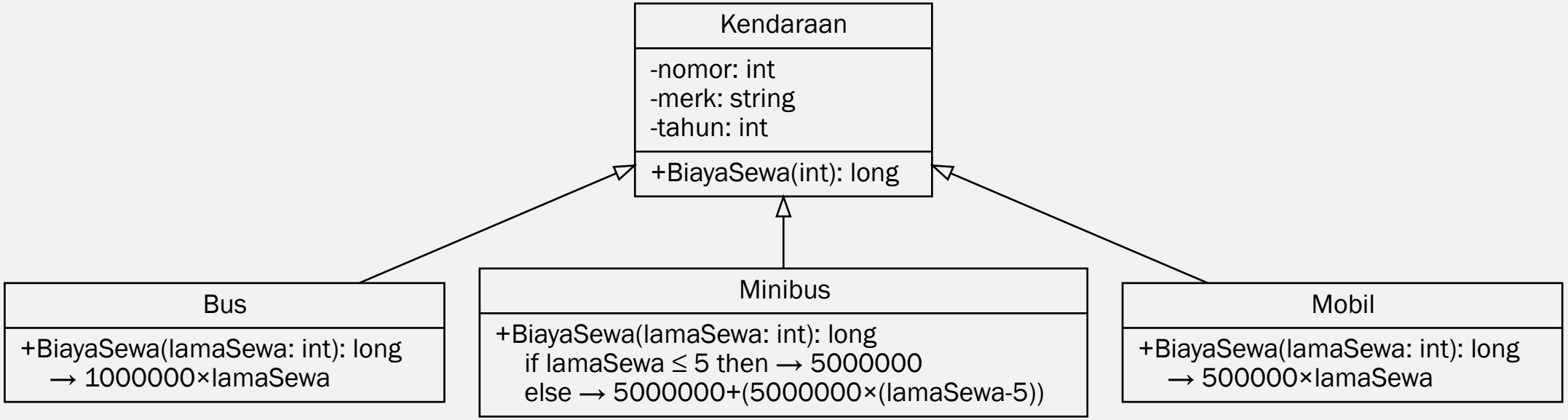
Inheritance dalam OOP

- Inheritance menyatakan bahwa sebuah kelas (disebutsebagai subclass) merepresentasikan **bentuk khusus** dari konsep yang lebih umum (superclass).
- Subclass / derived class:
  - mempertahankan makna dasar superclass / base class,
  - memenuhi kontrak perilaku yang sama,
  - dapat digunakan di semua situasi yang membutuhkan superclass.
- Inheritance digunakan untuk memodelkan **relasi konseptual "general → specific"** dalam domain.

Mengapa inheritance?

- Bayangkan contoh kelas `Kendaraan` dalam konteks persewaan kendaraan.
- Untuk perhitungan biaya sewa kendaraan terdapat kondisi:
  - Jika kategori="bus" maka biaya sewa = 1.000.000 × lama sewa
  - Jika kategori="minibus" maka biaya sewa = ...
  - Jika kategori="mobil" maka biaya sewa = ...
- Dalam paradigma prosedural, diimplementasikan dengan rangkaian if-else atau switch-case dalam fungsi `BiayaSewa(int)`.
- Dalam OO, kendaraan dengan kategori yang berbeda-beda ini dapat dirancang sebagai *subclass* dari kelas `Kendaraan`.

Kendaraan dengan Inheritance



Notasi: diagram kelas (UML) yang dimodifikasi untuk memudahkan ilustrasi (jangan ditiru)

- Tidak ada lagi atribut kategori.
- `BiayaSewa()` diimplementasikan di *subclass*.
- Efek: jika ada jenis kendaraan baru, dapat membuat *subclass* baru tanpa mengubah kelas `Kendaraan`.

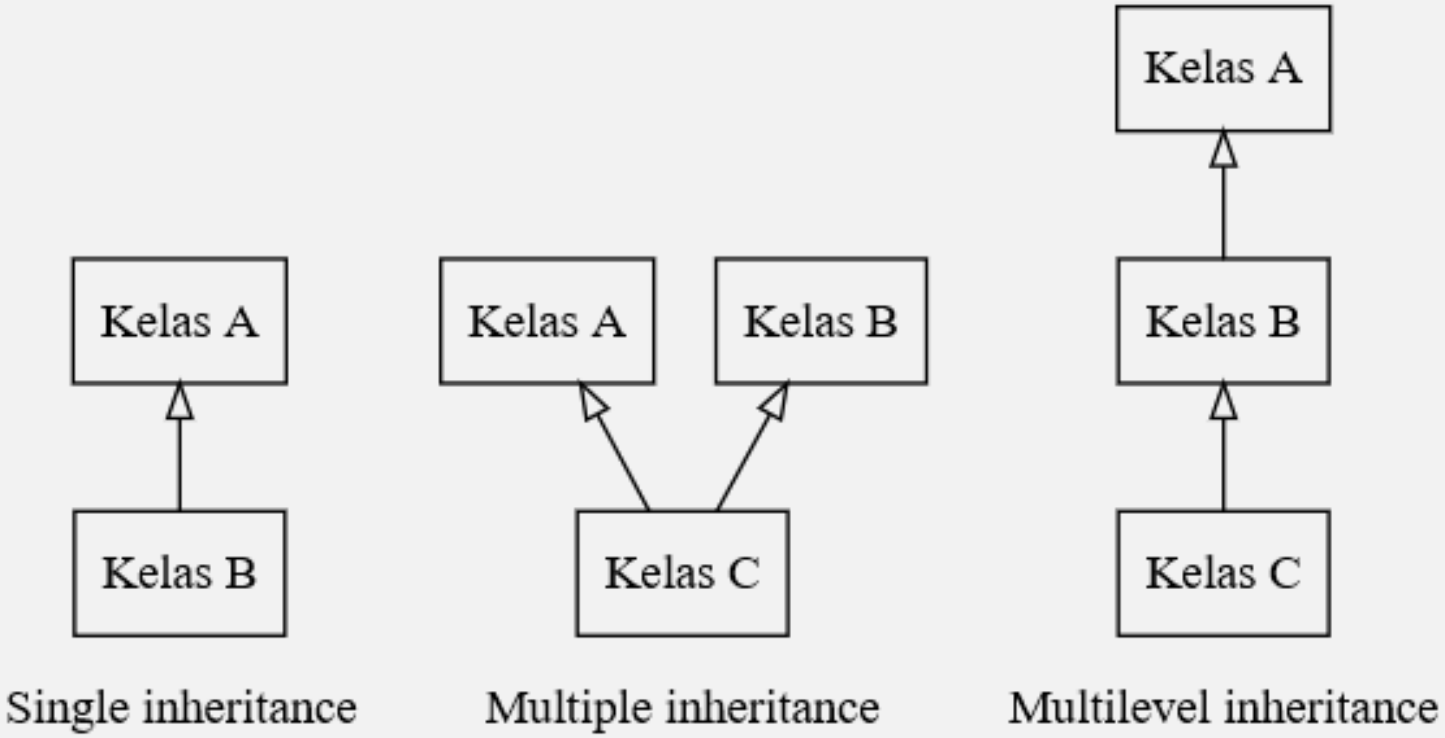
Inheritance untuk Memodelkan Variasi Perilaku

- Dalam suatu domain, konsep umum sering memiliki beberapa bentuk nyata.
- Pada contoh di atas, `Bus`, `Mobil`, dan `Minibus`:
  - tetap merupakan `Kendaraan` (memiliki kontrak yang sama),
  - tetapi memiliki aturan perilaku yang berbeda (mis. perhitungan biaya sewa).
- Inheritance memungkinkan sistem menyatakan bahwa:
  - semua objek berbagi identitas konseptual yang sama,
  - setiap objek menjalankan perilakunya sesuai bentuk spesifiknya.
- Dengan demikian, sistem bekerja melalui **substitusi objek**, bukan pengujian kategori.

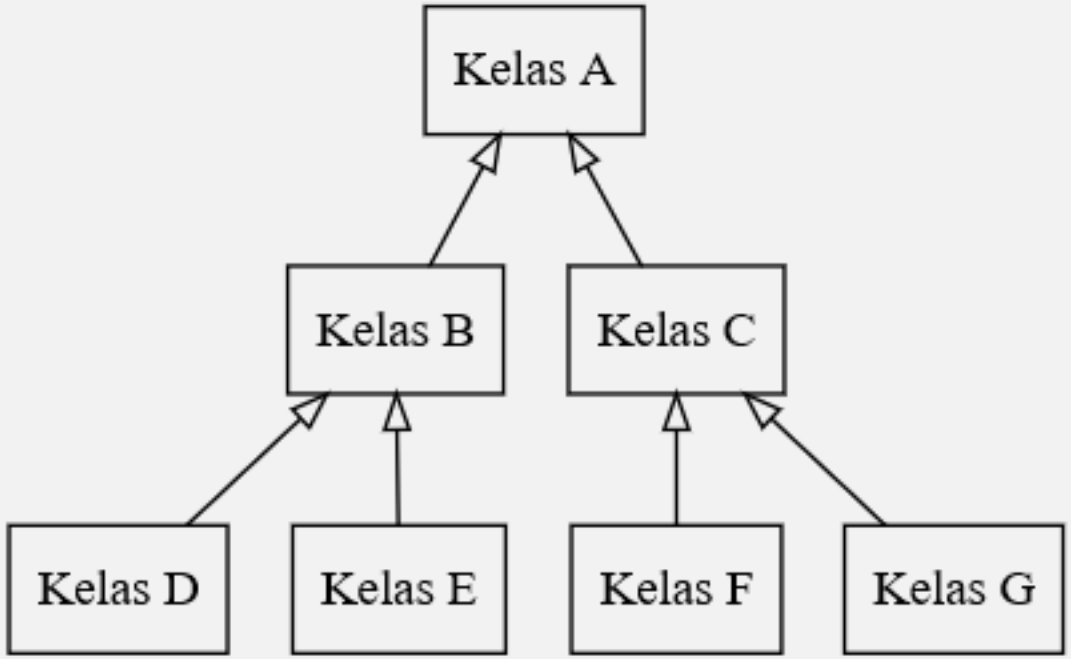
Kelas abstrak

- Pada contoh sebelumnya, kelas Kendaraan adalah kelas abstrak.
  - Ada method yang tidak diimplementasikan (`BiayaSewa(int)`).
- Kelas abstrak tidak dapat langsung diinstansiasi.
- Objek yang diciptakan adalah instansiasi dari subclass-nya yang tidak abstrak.
- Contoh lain: kelas `Shape` adalah abstrak, tidak bisa digambar (method `Draw()` tidak dapat diimplementasi) jika tidak tahu bentuk “nyata”-nya.
  - Subclass `Rectangle`, `Circle` diketahui bagaimana cara menggambarnya (mengimplementasikan `Draw()` sesuai atribut yang dimiliki: `Rectangle` memiliki panjang dan lebar, `Circle` memiliki titik pusat dan radius).

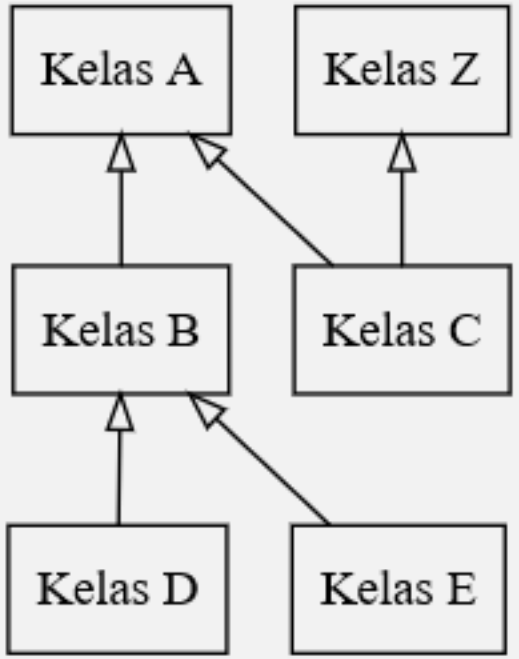
Jenis-jenis *inheritance* (1)



Jenis-jenis *inheritance* (2)



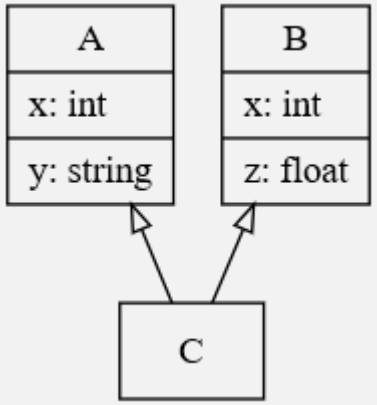
Hierarchical inheritance



Hybrid/virtual inheritance

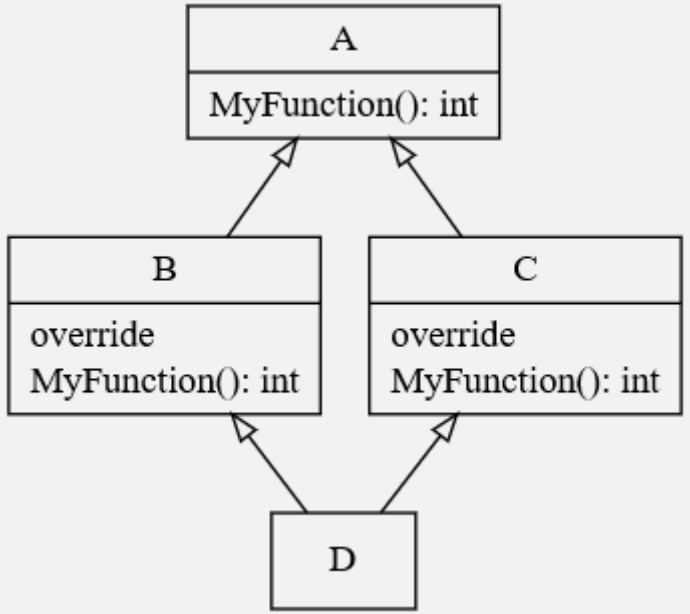
Masalah pada multiple inheritance (1)

- Beberapa *base class* memiliki nama anggota yang sama.



- `c.x` yang mana?

- *Diamond problem/deadly diamond of death* (DDD).



- `d.MyFunction()` yang mana?

Masalah pada multiple inheritance (2)

- Untuk mengatasi masalah-masalah tersebut, setiap bahasa OO memiliki caranya sendiri, misalnya:
  - Rename, redefinition
  - C++: untuk mencegah DDD, kelas B dan C harus dideklarasikan virtual
  - Java, Ruby, Smalltalk: tidak memperbolehkan *multiple inheritance*
  - dll.
- Ini sebenarnya persoalan akibat pemodelan domain yang tidak “bersih”…

Ketika Inheritance Salah Digunakan

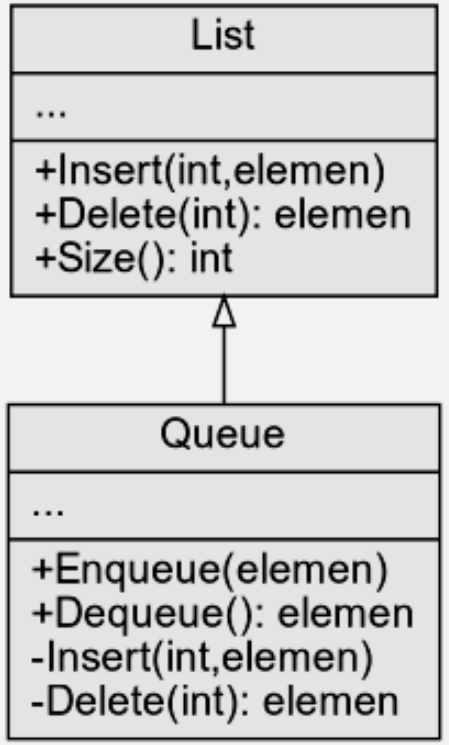
- Masalah seperti *diamond problem* sering dianggap masalah bahasa.
- Sebenarnya itu biasanya tanda kita mencoba menggabungkan beberapa sumber tanggung jawab yang tidak benar-benar satu konsep.
- Jika dua *superclass* tidak merepresentasikan satu abstraksi yang sama, tetapi kita tetap “menggabungkannya”,
  - maka desainnya yang keliru, bukan sintaksnya.
- Pertanyaan yang harus ditanya sebelum membuat *subclass*: Apakah objek ini benar-benar dapat menggantikan *superclass*-nya tanpa mengejutkan pengguna?
  - Jika tidak: gunakan ***composition***, bukan *inheritance*.

Inheritance vs. *Composition*

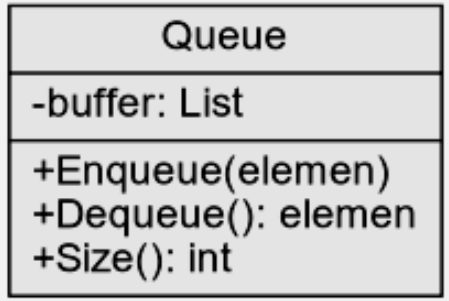
- *Inheritance*: hubungan *is-a*, *composition*: hubungan *has-a*.
  - Sedan *is a* mobil, SUV *is a* mobil, dst.
  - Mobil *has a* mesin, mesin *has a* set of roda gigi, dst.
- Misalkan kita memiliki sebuah kelas `List` yang bisa ditambah/kurangi elemennya di posisi manapun
  - Kita dapat memanfaatkan kelas tersebut untuk membuat kelas `Queue`:
    - Komposisi? (*Queue has a List*)
    - Inheritance? (*Queue is a List*)
      - `Queue` harus bisa melakukan apa pun yang bisa dilakukan `List` (seharusnya tidak)

Queue (is a|has a) List

Queue 'is a' List



Queue 'has a' List



- `Queue` harus menyembunyikan `Insert(int,elemen)` dan `Delete(int)`

- `Enqueue(e)` memanggil `buffer.Insert(last_idx,e)`
- `Dequeue()` mengembalikan `buffer.Delete(0)`
- `Size()` membungkus `buffer.Size()`

Root object/class

- Jika sebuah kelas dapat merupakan turunan dari kelas lain, adakah *“mother of all classes”*?
- Pada beberapa bahasa OO, semua kelas, baik yang disediakan oleh bahasa/*framework* maupun yang dibuat sendiri oleh pemrogram, adalah *subclass* dari sebuah *root class* meskipun tidak dituliskan secara eksplisit.
  - C#: kelas `System.Object`
  - Java: kelas `java.lang.Object`
  - Python: kelas `object`

Polymorphism

- Objek-objek dari kelas turunan memiliki sifat sebagai kelas tersebut dan sekaligus *superclass*-nya.
  - *polymorphism* (*poly* = banyak, *morph* = bentuk).
- Sebuah "variabel" dapat dideklarasikan sebagai kelas `Kendaraan`, namun pada *run time* dapat merupakan instansiasi dari kelas `Mobil` atau `Bus` (*subclass* dari `Kendaraan`).
- Sebaliknya, sebuah "variabel" bertipe `Mobil` dapat diperlakukan sebagai `Kendaraan`, e.g., di-passing ke method yg menerima parameter `Kendaraan`.
  - i.e. *message* yang dapat diterima `Kendaraan` juga dapat diterima oleh `Mobil`.
- Hubungan "is-a": `Mobil` is a `Kendaraan`, `Bus` is a `Kendaraan`.

Takeaway

- *Inheritance* digunakan untuk menyatakan bahwa beberapa objek merupakan bentuk-bentuk khusus dari satu konsep yang sama,
  dan dapat diperlakukan melalui pemahaman umum tentang konsep tersebut.
- Dengan *inheritance*, kita:
  - mengorganisasi model berdasarkan relasi ***general → specific***,
  - memungkinkan variasi perilaku muncul secara alami dari jenis objeknya,
  - bekerja melalui **substitusi objek**, bukan pemeriksaan tipe atau kategori,
  - menjaga konsistensi makna di seluruh hierarki konsep.
- *Inheritance* paling tepat digunakan ketika relasi antar kelas benar-benar merepresentasikan **spesialisasi dalam domain**, sehingga setiap *subclass* tetap "bermakna sebagai" *superclass*-nya dalam semua konteks penggunaan.